9/4/2026

PROGRAMMING 2B PROG6212

Seaney Maseko ST10493031

Contents

Section A ERD 2

Section B: API Endpoint Plan 5

Section C: SQL Database Script 11

GitHub and Reference 21

Part 2 23

VS 23

GitHUB 24

Read ME 25

Section A ERD

Entity

Users

- Stores each user account and links the user to a role such as Organiser or Participant.

Roles

- Lookup table for the two system roles: Organiser and Participant.

Events

- Stores event information including name, description, date, location, distance and event type.

Categories

- Stores age-based or distance-based categories used for event participation.

EventCategories

- Junction table connecting Events and Categories in a many-to-many relationship.

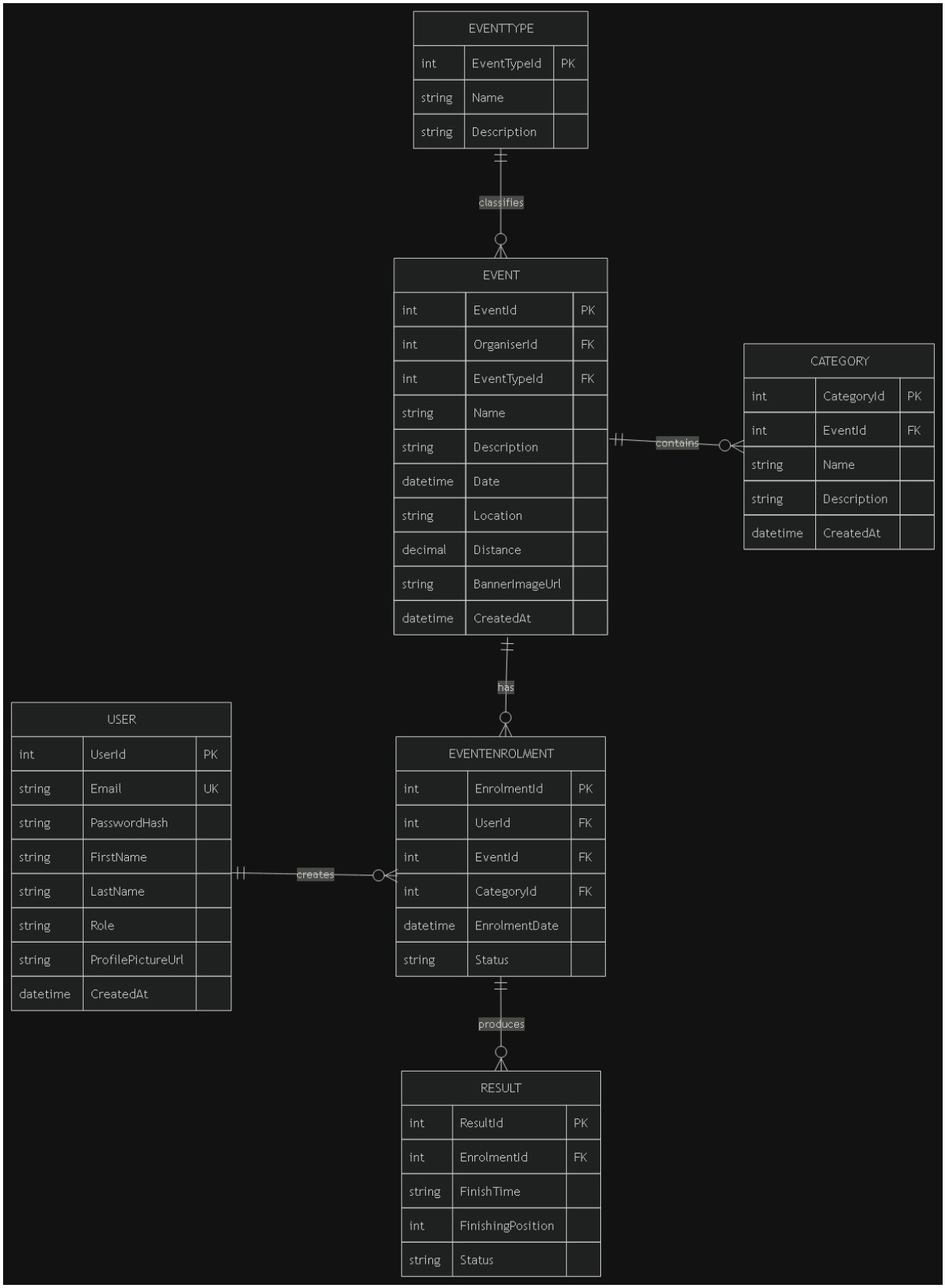
Enrolments

- Records Participants entering Events and selecting a Category.

Results

- Stores a Participant result after an event, including finish time and finishing position.

ERD


EVENTTYPE
int EventTypeId PK
string Name
string Description
classifies
EVENT
int EventId PK
int OrganiserId FK
int EventTypeId FK
string Name
string Description
datetime Date
string Location
decimal Distance
string BannerImageUrl
datetime CreatedAt
contains
CATEGORY
int CategoryId PK
int EventId FK
string Name
string Description
datetime CreatedAt
has
USER
int UserId PK
string Email UK
string PasswordHash
string FirstName
string LastName
string Role
string ProfilePictureUrl
datetime CreatedAt
creates
EVENTENROLMENT
int EnrolmentId PK
int UserId FK
int EventId FK
int CategoryId FK
datetime EnrolmentDate
string Status
produces
RESULT
int ResultId PK
int EnrolmentId FK
string FinishTime
int FinishingPosition
string Status

(draw.io)

Relationships

- Users - Roles: Each user has one role. A role can have many users.
- Users - Enrolments: One user (participant) can have many enrolments. Each enrolment belongs to one user.
- Users - Events: One user (organiser) can create many events. Each event has one organiser.
- Events - Enrolments: One event can have many enrolments. Each enrolment is for one event.
- Events - Categories: Connected through a middle table (EventCategories) because events and categories have a many-to-many relationship.
- Enrolments Results: Each enrolment has zero or one result. Each result belongs to one enrolment.
- Enrolments - Categories: Each enrolment picks one category. A category can be used by many enrolments.

Primary and Foreign Key Constraint Summary

- Users: PK = UserId; FK = RoleId – references Roles.RoleId.
- Roles: PK = RoleId; no foreign keys.
- Events: PK = EventId; FK = OrganiserId – references Users.UserId.
- Categories: PK = CategoryId; no foreign keys.
- EventCategories: PK = EventCategoryId; FKs = EventId - Events.EventId and CategoryId - Categories.CategoryId.
- Enrolments: PK = EnrolmentId; FKs = ParticipantId - Users.UserId, EventId - Events.EventId, and CategoryId - Categories.CategoryId.
- Results: PK =ResultId; FK =EnrolmentId - Enrolments.EnrolmentId (with a unique constraint to maintain the One-to-One cardinality).

Section B: API Endpoint Plan

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Register a new user account with role selection	None (Public)	{ email, password, firstName, lastName, role }	201 Created - user details 400 Bad Request - validation errors 409 Conflict - email exists
POST	/api/auth/login	Authenticate user and create session	None (Public)	{ email, password }	200 OK - { message, role, userId } 401 Unauthorized - invalid credentials
POST	/api/auth/logout	End current user session	Any (Logged in)	None	200 OK - logged out successfully
GET	/api/users/profile	Get current user's profile information	Any (Logged in)	None	200 OK - user details 401 Unauthorized

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
PUT	/api/users/profile	Update current user's profile	Any (Logged in)	{ firstName, lastName, profilePictureUrl }	200 OK - updated user 400 Bad Request - validation errors 401 Unauthorized
GET	/api/events	Get all events with optional filters	None (Public)	None	200 OK - list of events
GET	/api/events/{id}	Get specific event details	None (Public)	None	200 OK - event details 404 Not Found
POST	/api/events	Create a new event	Organiser	{ name, description, date, location, distance, eventTypeId, bannerImageUrl }	201 Created - new event 400 Bad Request 401 Unauthorized
PUT	/api/events/{id}	Update an existing event	Organiser	{ name, description, date, location, distance, eventTypeId, bannerImageUrl }	200 OK - updated event 400 Bad Request 401 Unauthorized 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
DELETE	/api/events/{id}	Delete an event	Organiser	None	204 No Content 401 Unauthorized 404 Not Found
GET	/api/events/{eventId}/categories	Get all categories for an event	None (Public)	None	200 OK - list of categories 404 Not Found
POST	/api/events/{eventId}/categories	Create a new category for an event	Organiser	{ name, description }	201 Created - new category 400 Bad Request 401 Unauthorized
PUT	/api/categories/{id}	Update a category	Organiser	{ name, description }	200 OK - updated category 401 Unauthorized 404 Not Found
DELETE	/api/categories/{id}	Delete a category	Organiser	None	204 No Content 401 Unauthorized 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/enrolments	Get all enrolments for an event	Organiser	None	200 OK - list of enrolments 401 Unauthorized 404 Not Found
POST	/api/events/{eventId}/enrol	Enrol a participant in an event	Participant	{ categoryId }	201 Created - enrolment details 400 Bad Request 401 Unauthorized 409 Conflict
GET	/api/users/enrolments	Get all enrolments for the current user	Participant	None	200 OK - list of enrolments 401 Unauthorized
PUT	/api/enrolments/{id}/status	Update enrolment status	Organiser	{ status }	200 OK - updated enrolment 401 Unauthorized 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments/{enrolmentId}/results	Capture result for a participant	Organiser	{ finishTime, finishingPosition }	201 Created - result details 400 Bad Request 401 Unauthorized
GET	/api/users/results	Get current user's race results	Participant	None	200 OK - list of results 401 Unauthorized
GET	/api/events/{eventId}/results	Get all results for an event	Organiser	None	200 OK - list of results 401 Unauthorized 404 Not Found
GET	/api/eventtypes	Get all event types (run/walk/cycle)	None (Public)	None	200 OK - list of event types
POST	/api/events/{id}/banner	Upload event banner image	Organiser	multipart/form-data file	200 OK - { bannerImageUrl } 401 Unauthorized 404 Not Found

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
PUT	/api/users/profile/picture	Upload profile picture	Any (Logged in)	multipart/form-data file	200 OK - { profilePictureUrl } 401 Unauthorized

I

Section C: SQL Database Script

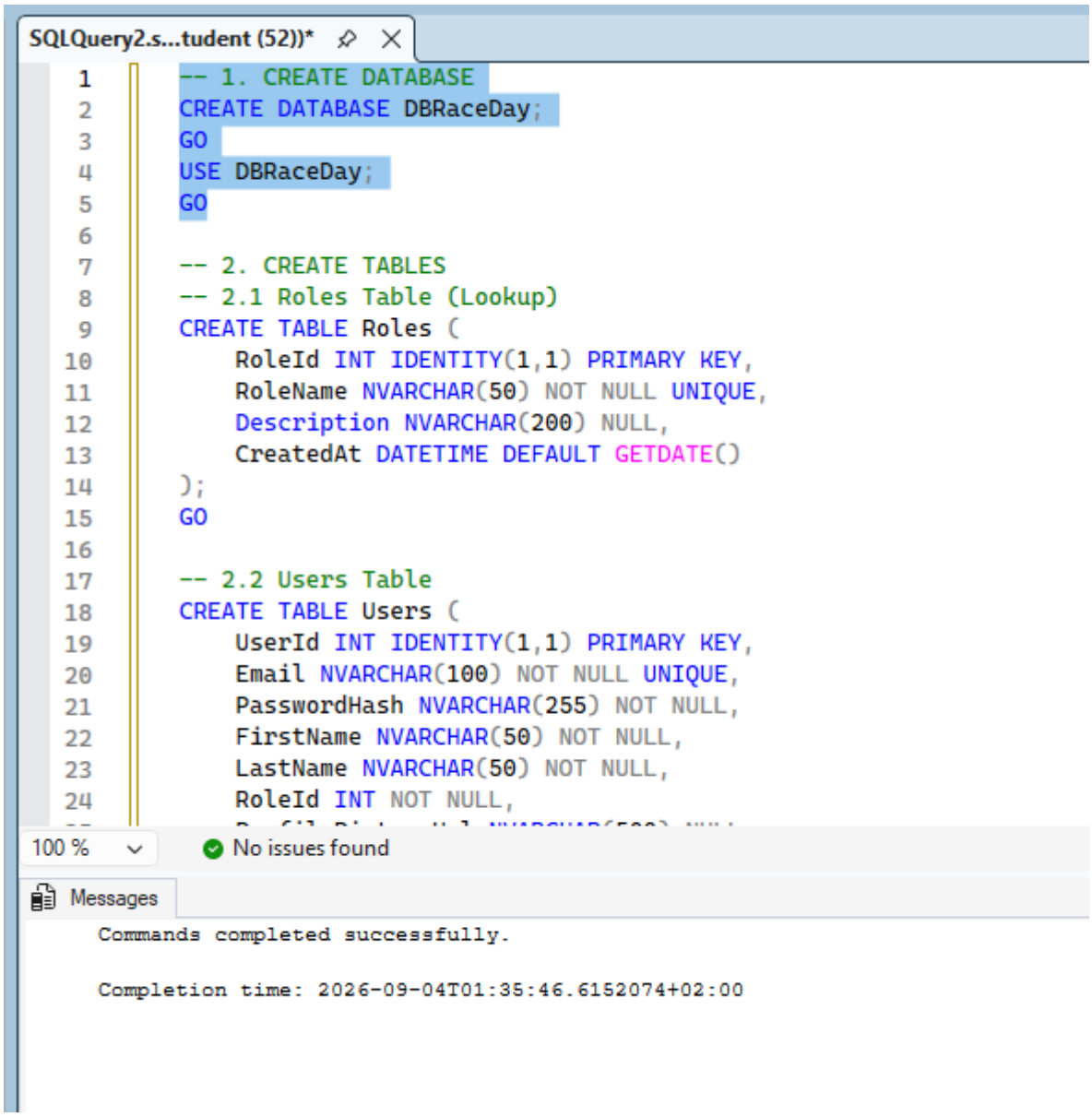
-- 1. CREATE DATABASE

CREATE DATABASE DBRaceDay;

GO

USE DBRaceDay;

GO



-- 2.1. Roles Table

CREATE TABLE Roles (

RoleId INT IDENTITY(1,1) PRIMARY KEY,

```
    RoleName NVARCHAR(50) NOT NULL UNIQUE,
    Description NVARCHAR(200) NULL,
    CreatedAt DATETIME DEFAULT GETDATE()
);
GO

-- 2.2 Users Table
CREATE TABLE Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Email NVARCHAR(100) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    RoleId INT NOT NULL,
    ProfilePictureUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Users_Roles FOREIGN KEY (RoleId) REFERENCES Roles(RoleId),
    CONSTRAINT CHK_Users_Email CHECK (Email LIKE '%_@__%.__%')
);
GO

-- 2.3. EventType Table (NEW – matches ERD)
CREATE TABLE EventType (
    EventTypeId INT IDENTITY(1,1) PRIMARY KEY,
    TypeName NVARCHAR(20) NOT NULL UNIQUE CHECK (TypeName IN ('Run', 'Walk',
'Cycle')),
    Description NVARCHAR(200) NULL,
    CreatedAt DATETIME DEFAULT GETDATE()
```

```
);
GO

-- 2.4. Events Table
CREATE TABLE Events (
    EventId INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserId INT NOT NULL,
    EventTypeId INT NOT NULL,                    -- changed
    Name NVARCHAR(100) NOT NULL,
    Description NVARCHAR(500) NULL,
    EventDate DATETIME NOT NULL,
    Location NVARCHAR(200) NOT NULL,
    Distance DECIMAL(8,2) NOT NULL,
    BannerImageUrl NVARCHAR(500) NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Events_Users FOREIGN KEY (OrganiserId) REFERENCES Users(UserId),
    CONSTRAINT FK_Events_EventType FOREIGN KEY (EventTypeId) REFERENCES
EventType(EventTypeId)
);
GO

-- 2.5. Categories Table
CREATE TABLE Categories (
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(50) NOT NULL,
    Description NVARCHAR(200) NULL,
    CategoryType NVARCHAR(20) NOT NULL CHECK (CategoryType IN ('Age', 'Distance')),
    CreatedAt DATETIME DEFAULT GETDATE()
```

```
);
GO
-- 2.6. EventCategories
CREATE TABLE EventCategories (
    EventCategoryId INT IDENTITY(1,1) PRIMARY KEY,
    EventId INT NOT NULL,
    CategoryId INT NOT NULL,
    CONSTRAINT FK_EventCategories_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON DELETE CASCADE,
    CONSTRAINT FK_EventCategories_Categories FOREIGN KEY (CategoryId) REFERENCES Categories(CategoryId) ON DELETE CASCADE,
    CONSTRAINT UQ_EventCategories_EventCategory UNIQUE (EventId, CategoryId)
);
GO

-- 2.7. Enrolments Table
CREATE TABLE Enrolments (
    EnrolmentId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    EventId INT NOT NULL,
    CategoryId INT NOT NULL,
    EnrolmentDate DATETIME DEFAULT GETDATE(),
    Status NVARCHAR(20) NOT NULL DEFAULT 'Pending'
        CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled', 'Completed')),
    CONSTRAINT FK_Enrolments_Users FOREIGN KEY (ParticipantId) REFERENCES Users(UserId),
    CONSTRAINT FK_Enrolments_Events FOREIGN KEY (EventId) REFERENCES Events(EventId) ON DELETE CASCADE,
```

```
    CONSTRAINT FK_Enrolments_Categories FOREIGN KEY (CategoryId) REFERENCES
Categories(CategoryId),
    CONSTRAINT UQ_Enrolments_ParticipantEvent UNIQUE (ParticipantId, EventId)
);
GO

-- 2.8. Results Table
CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
    EnrolmentId INT NOT NULL,
    FinishTime TIME NOT NULL,
    FinishingPosition INT NOT NULL,
    Status NVARCHAR(20) NOT NULL DEFAULT 'Finished'
        CHECK (Status IN ('DNS', 'DNF', 'Finished')),
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT FK_Results_Enrolments FOREIGN KEY (EnrolmentId) REFERENCES
Enrolments(EnrolmentId) ON DELETE CASCADE,
    CONSTRAINT UQ_Results_Enrolment UNIQUE (EnrolmentId)
);
GO

-- Indexes for performance
CREATE INDEX IX_Users_RoleId ON Users(RoleId);
CREATE INDEX IX_Users_Email ON Users(Email);
CREATE INDEX IX_Events_OrganiserId ON Events(OrganiserId);
CREATE INDEX IX_Events_EventDate ON Events(EventDate);
CREATE INDEX IX_EventCategories_EventId ON EventCategories(EventId);
CREATE INDEX IX_EventCategories_CategoryId ON EventCategories(CategoryId);
CREATE INDEX IX_Enrolments_ParticipantId ON Enrolments(ParticipantId);
```

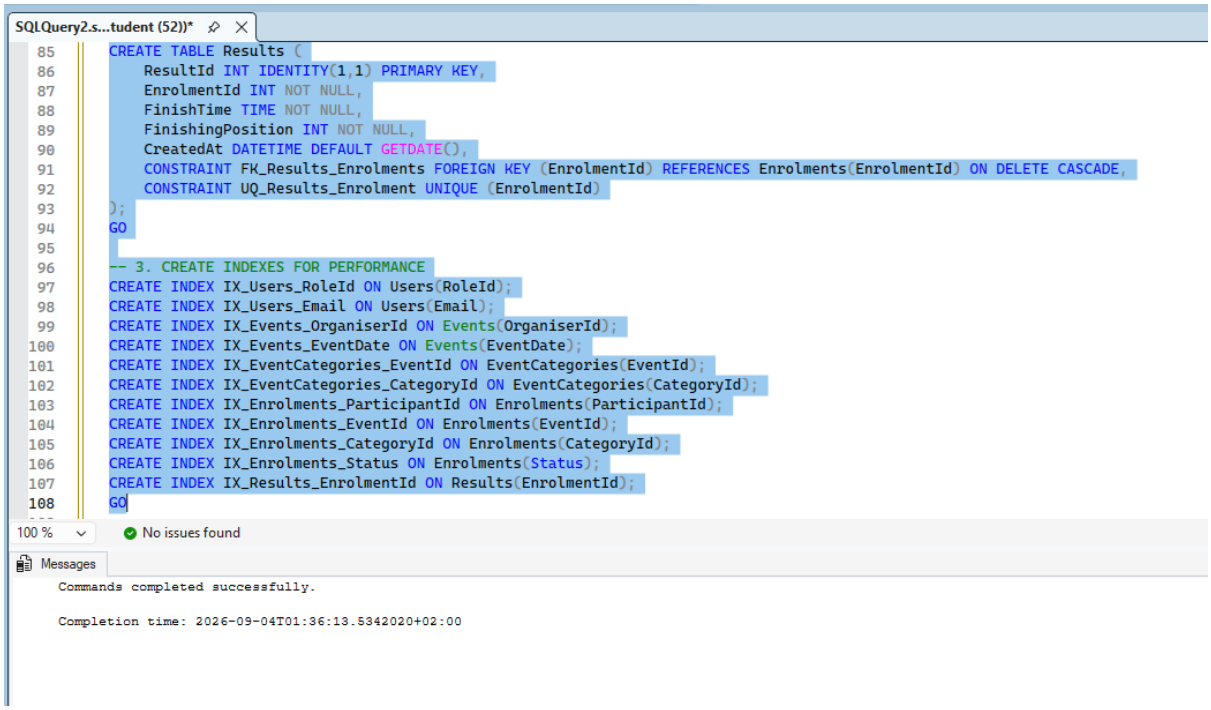
CREATE INDEX IX_Enrolments_EventId ON Enrolments(EventId);

CREATE INDEX IX_Enrolments_CategoryId ON Enrolments(CategoryId);

CREATE INDEX IX_Enrolments_Status ON Enrolments(Status);

CREATE INDEX IX_Results_EnrolmentId ON Results(EnrolmentId);

GO



-- 4. SEED DATA

-- 1. Roles

INSERT INTO Roles (RoleName, Description) VALUES

('Organiser', 'Event organiser who can create and manage events, categories, and results'),

('Participant', 'Event participant who can browse events, enrol, and view results');

GO

-- 2. Event Types

INSERT INTO EventType (TypeName, Description) VALUES

('Run', 'Road running events including marathons, half-marathons, and fun runs'),

('Walk', 'Walking events including park runs and charity walks'),

('Cycle', 'Cycling events including road races and time trials');

```
GO

-- 4.3. Users (passwords hashed – placeholders)
INSERT INTO Users (Email, PasswordHash, FirstName, LastName, RoleId, ProfilePictureUrl) VALUES
('thabo.mokoena@raceday.co.za', 'hashed_password_1', 'Thabo', 'Mokoena', 1, NULL),
('linda.zulu@raceday.co.za', 'hashed_password_2', 'Linda', 'Zulu', 1, NULL),
('sipho.ndlovu@gmail.com', 'hashed_password_3', 'Sipho', 'Ndlovu', 2, NULL),
('zanele.dlamini@gmail.com', 'hashed_password_4', 'Zanele', 'Dlamini', 2, NULL);
GO

-- 4.4. Categories
INSERT INTO Categories (Name, Description, CategoryType) VALUES
('Under 20', 'Participants aged under 20 years', 'Age'),
('Senior (20-39)', 'Participants aged 20 to 39 years', 'Age'),
('Master (40+)', 'Participants aged 40 years and over', 'Age'),
('10km', '10 kilometre distance category', 'Distance'),
('21km', '21.1 kilometre half-marathon category', 'Distance'),
('42km', '42.2 kilometre marathon category', 'Distance');
GO

-- 4.5. Events (now using EventTypeId)
INSERT INTO Events (OrganiserId, EventTypeId, Name, Description, EventDate, Location, Distance, BannerImageUrl) VALUES
(1, 1, 'Soweto Marathon', 'Annual Soweto Marathon through the streets of Soweto', '2026-09-15 06:00:00', 'Soweto, Johannesburg', 42.20, NULL),
(1, 3, 'Cape Town Cycle Tour', 'World-famous Cape Town Cycle Tour', '2026-10-20 07:00:00', 'Cape Town', 109.00, NULL),
```

```
(2, 1, 'Two Oceans Marathon', 'Ultra-marathon with stunning coastal views', '2026-11-05 05:30:00', 'Cape Town', 56.00, NULL);

GO


-- 4.6. Link Events to Categories

INSERT INTO EventCategories (EventId, CategoryId) VALUES

(1, 1), (1, 2), (1, 3), (1, 6),

(2, 1), (2, 2), (2, 3),

(3, 1), (3, 2), (3, 3), (3, 5);

GO


-- 4.7. Enrolments

INSERT INTO Enrolments (ParticipantId, EventId, CategoryId, Status) VALUES

(3, 1, 2, 'Confirmed'),

(3, 2, 1, 'Pending'),

(4, 1, 3, 'Confirmed'),

(4, 3, 2, 'Pending');

GO

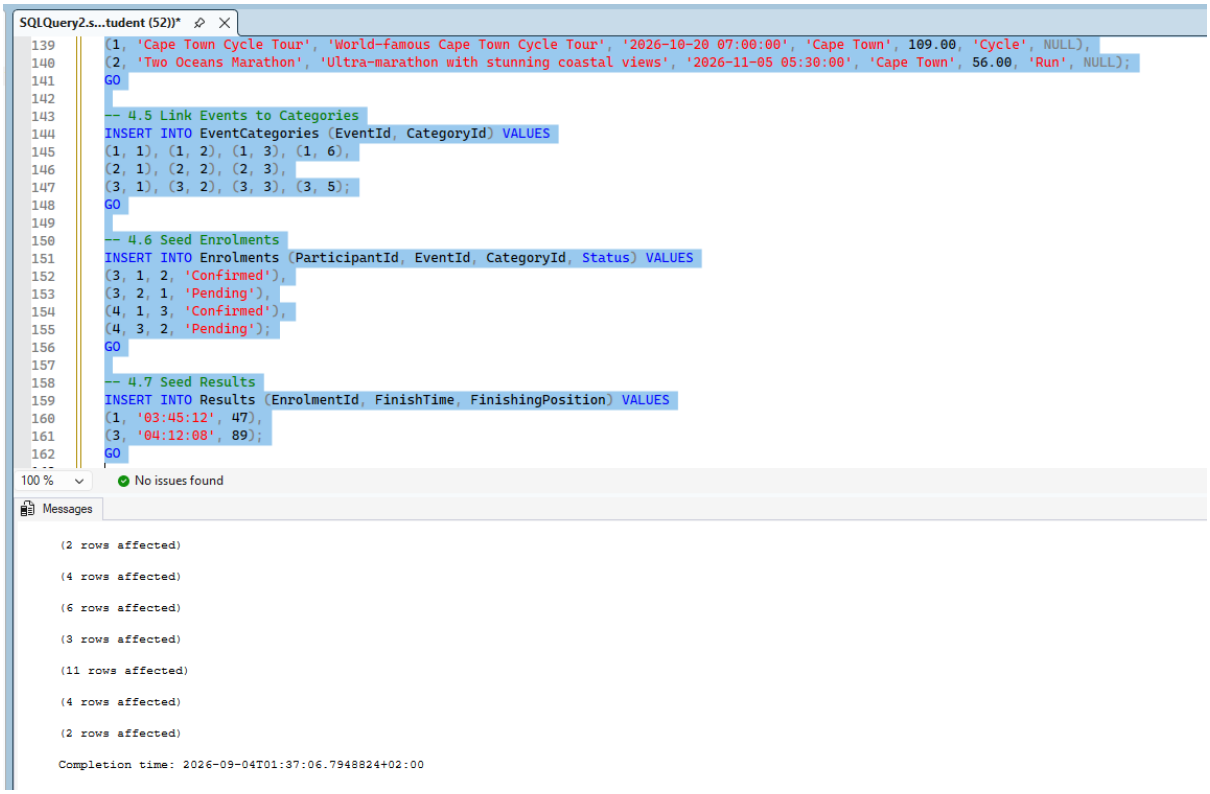

-- 4.8. Results (now with Status)

INSERT INTO Results (EnrolmentId, FinishTime, FinishingPosition, Status) VALUES

(1, '03:45:12', 47, 'Finished'),

(3, '04:12:08', 89, 'Finished');

GO
```



-- 5. VERIFICATION QUERIES

SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE';

GO

SELECT * FROM Roles;

SELECT * FROM EventType;

SELECT * FROM Users;

SELECT * FROM Categories;

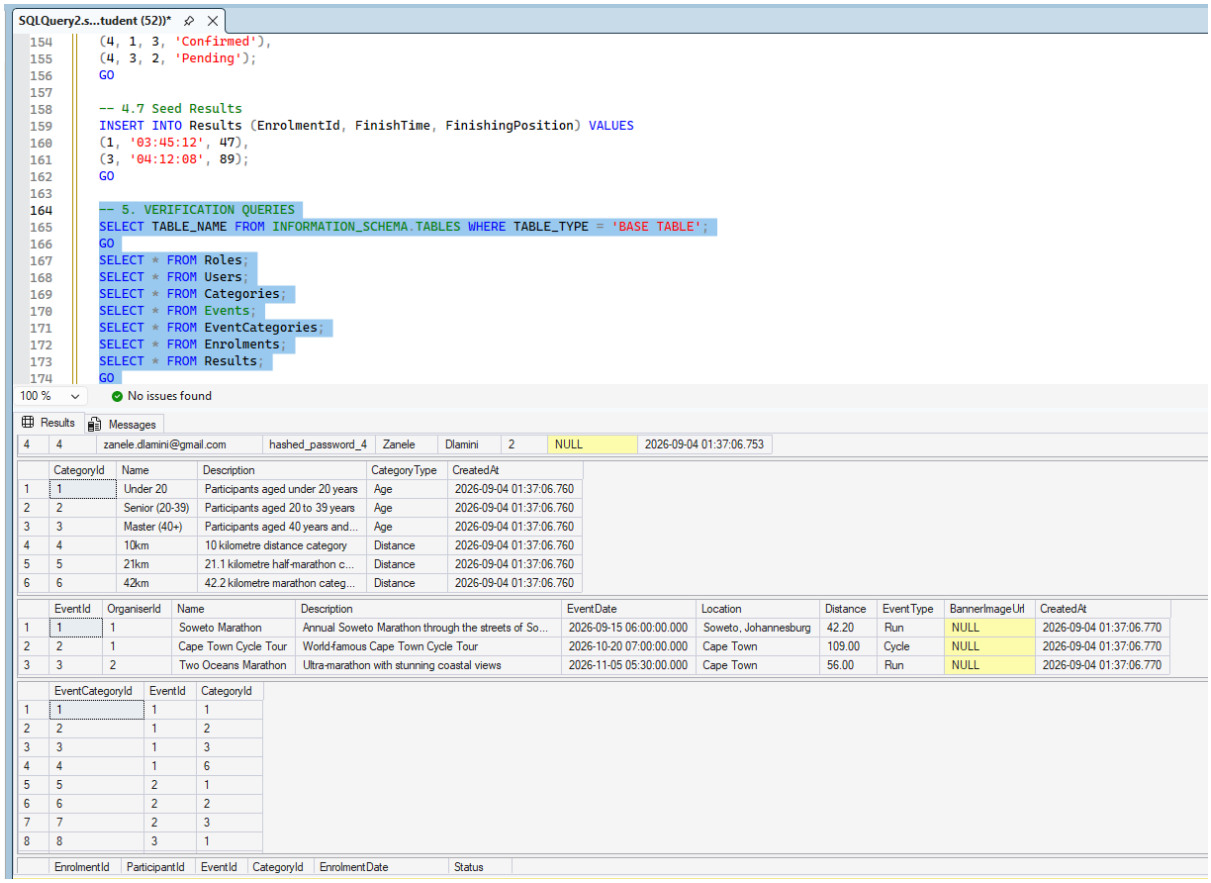
SELECT * FROM Events;

SELECT * FROM EventCategories;

SELECT * FROM Enrolments;

SELECT * FROM Results;

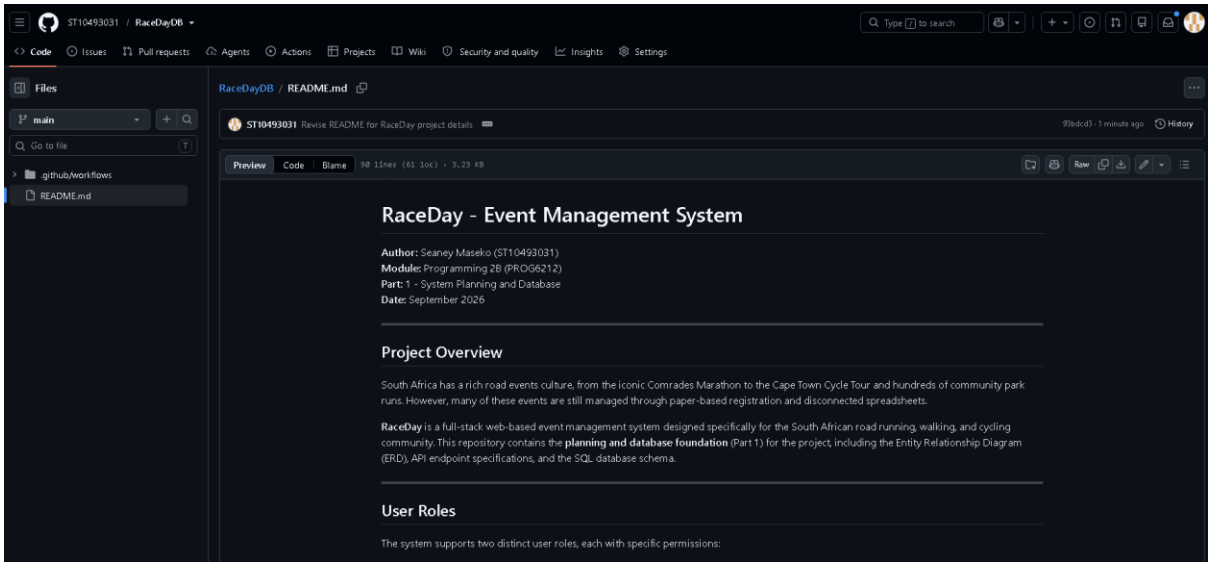
GO


SQLQuery2.s...tudent (52))*
(4, 1, 3, 'Confirmed'),
(4, 3, 2, 'Pending');
GO
-- 4.7 Seed Results
INSERT INTO Results (EnrolmentId, FinishTime, FinishingPosition) VALUES
(1, '03:45:12', 47),
(3, '04:12:08', 89);
GO
-- 5. VERIFICATION QUERIES
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE';
GO
SELECT * FROM Roles;
SELECT * FROM Users;
SELECT * FROM Categories;
SELECT * FROM Events;
SELECT * FROM EventCategories;
SELECT * FROM Enrolments;
SELECT * FROM Results;
GO
100 %
No issues found
Results
Messages
4 4 zanele.dlamini@gmail.com hashed_password_4 Zanele Dlamini 2 NULL 2026-09-04 01:37:06.753
CategoryId Name Description CategoryType CreatedAt
1 1 Under 20 Participants aged under 20 years Age 2026-09-04 01:37:06.760
2 2 Senior (20-39) Participants aged 20 to 39 years Age 2026-09-04 01:37:06.760
3 3 Master (40+) Participants aged 40 years and... Age 2026-09-04 01:37:06.760
4 4 10km 10 kilometre distance category Distance 2026-09-04 01:37:06.760
5 5 21km 21.1 kilometre half-marathon c... Distance 2026-09-04 01:37:06.760
6 6 42km 42.2 kilometre marathon categ... Distance 2026-09-04 01:37:06.760
EventId OrganiserId Name Description EventDate Location Distance EventType BannerImageUrl CreatedAt
1 1 1 Soweto Marathon Annual Soweto Marathon through the streets of So... 2026-09-15 06:00:00.000 Soweto, Johannesburg 42.20 Run NULL 2026-09-04 01:37:06.770
2 2 1 Cape Town Cycle Tour World-famous Cape Town Cycle Tour 2026-10-20 07:00:00.000 Cape Town 109.00 Cycle NULL 2026-09-04 01:37:06.770
3 3 2 Two Oceans Marathon Ultra-marathon with stunning coastal views 2026-11-05 05:30:00.000 Cape Town 56.00 Run NULL 2026-09-04 01:37:06.770
EventCategoryId EventId CategoryId
1 1 1 1
2 2 1 2
3 3 1 3
4 4 1 6
5 5 2 1
6 6 2 2
7 7 2 3
8 8 3 1
EnrolmentId ParticipantId EventId CategoryId EnrolmentDate Status

GitHub and Reference

https://github.com/ST10493031/RaceDayDB

Readme



Scr

Video
link

References

3Wschool (2025). W3Schools.Com. [online] W3schools.com. Available at: https://www.w3schools.com/sql/ [Accessed 28 Aug. 2026].

Bootstrap (2023). Get started with Bootstrap. [online] getbootstrap.com. Available at: https://getbootstrap.com/docs/5.3/getting-started/introduction/ [Accessed 28 Aug. 2026].

chcomley (2026a). Azure Boards documentation. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/devops/boards/ [Accessed 28 Aug. 2026].

chcomley (2026b). Azure DevOps documentation. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/devops/ [Accessed 28 Aug. 2026].

dimitri-furman (2024). Monitoring and performance tuning — Azure SQL Database & Azure SQL Managed Instance. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/azure-sql/database/monitor-tune-overview [Accessed 28 Aug. 2026].

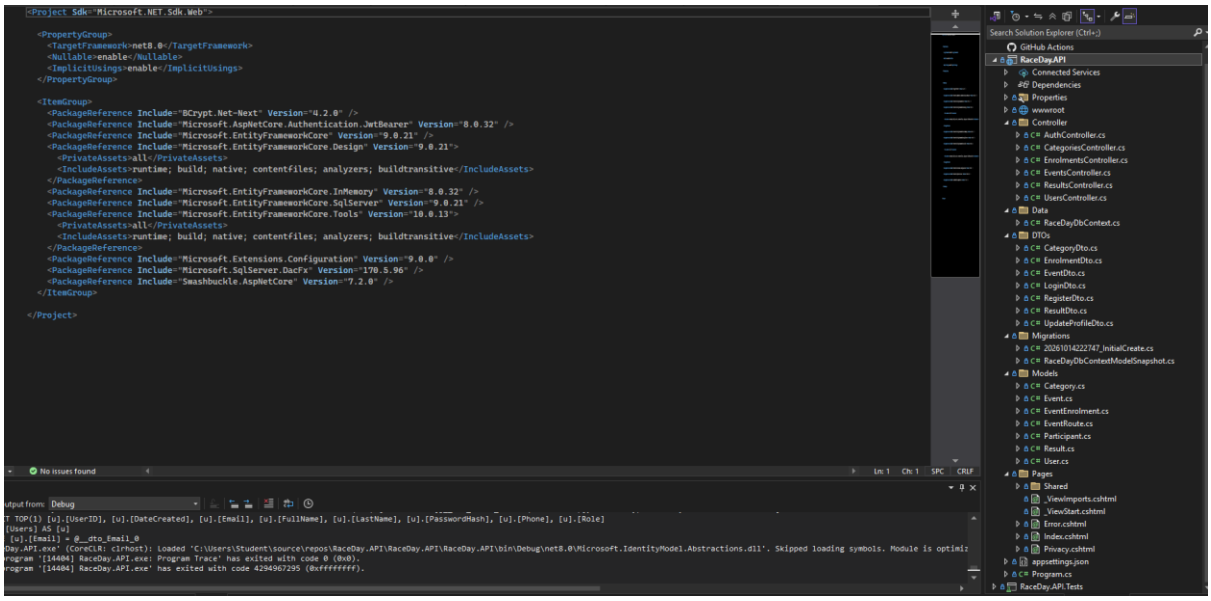
IBM (2024). Think topics | IBM. [online] IBM.com. Available at: https://www.ibm.com/think/topics/agile [Accessed 28 Aug. 2026].

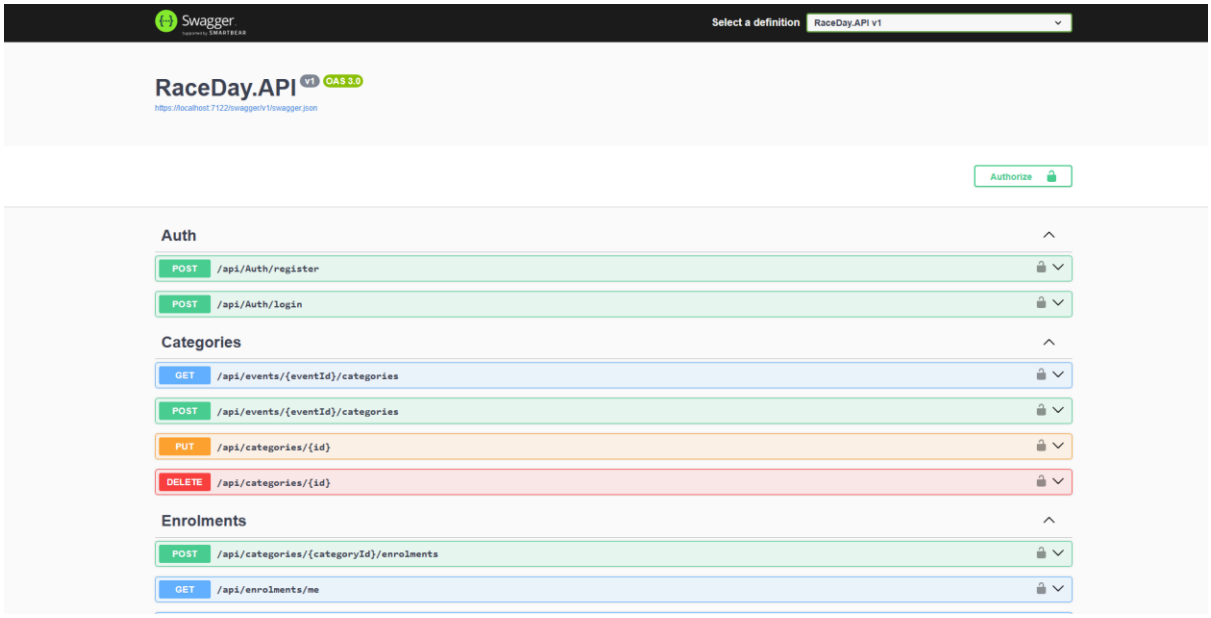
pauljewellmsft (2023). Get started with Azure Blob Storage and .NET — Azure Storage. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-dotnet-get-started [Accessed 3 Sept. 2026].

scrumguides (2025). Home | Scrum Guides. [online] scrumguides.org. Available at: https://scrumguides.org/ [Accessed 28 Aug. 2026].

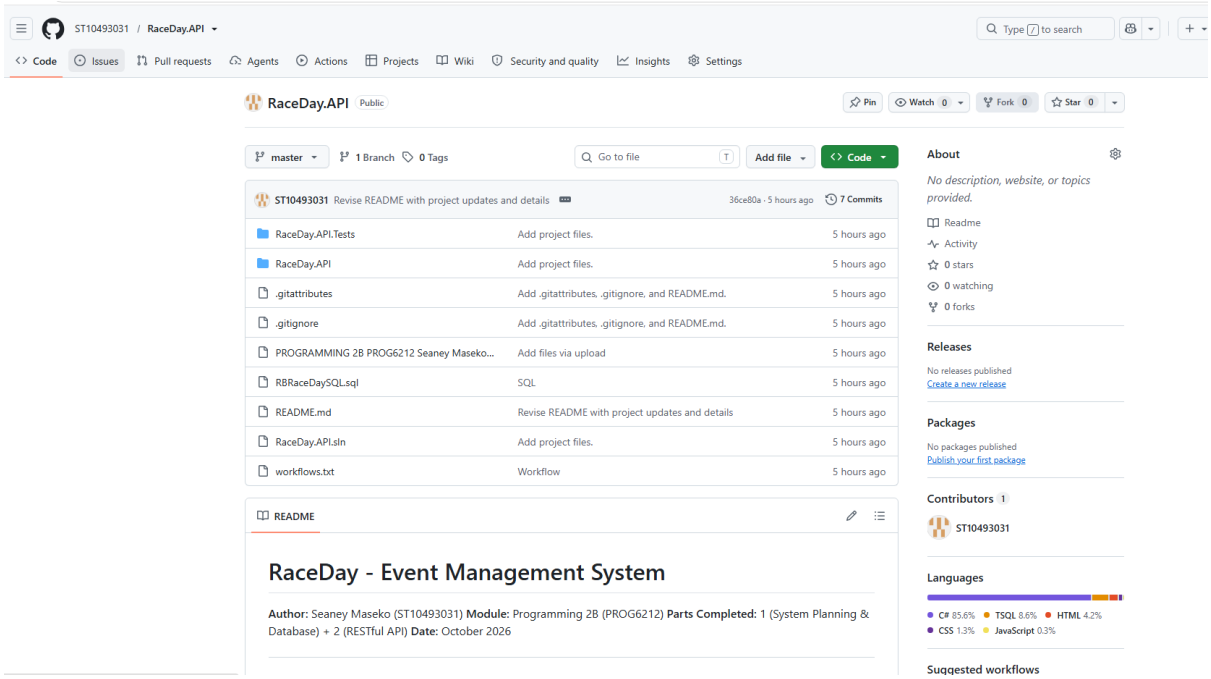
Part 2

VS


Project Sdk="Microsoft.NET.Sdk.Web"
TargetFramework>net8.0
Nullable>enable
ImplicitUsings>enable
ItemGroup
PackageReference Include="BCrypt.Net-Next" Version="4.2.0"
PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.32"
PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.21"
PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.21"
PrivateAssets>all
IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="8.0.32"
PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.21"
PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.13"
PackageReference Include="Microsoft.Extensions.Configuration" Version="9.0.0"
PackageReference Include="Microsoft.SqlServer.DacFx" Version="170.5.96"
PackageReference Include="Swashbuckle.AspNetCore" Version="7.2.0"
Project
No issues found
Search Solution Explorer (Ctrl+;)
GitHub Actions
RaceDayAPI
Connected Services
Dependencies
Properties
wwwroot
Controller
AuthController.cs
CategoriesController.cs
EnrolmentsController.cs
EventsController.cs
ResultsController.cs
UsersController.cs
Data
RaceDayDbContext.cs
DTOs
CategoryDto.cs
EnrolmentDto.cs
EventDto.cs
LoginDto.cs
RegisterDto.cs
ResultDto.cs
UpdateProfileDto.cs
Migrations
20261014222747_InitialCreate.cs
RaceDayDbContextModelSnapshot.cs
Models
Category.cs
Event.cs
EventEnrolment.cs
EventRoute.cs
Participant.cs
Result.cs
User.cs
Pages
Shared
_ViewImports.cshtml
_ViewStart.cshtml
Error.cshtml
Index.cshtml
Privacy.cshtml
appsettings.json
Program.cs
RaceDay.API.Tests


Swagger
Select a definition
RaceDay.API v1
RaceDay.API
v1
OAS 3.0
https://localhost:7122/swagger/v1/swagger.json
Authorize
Auth
POST /api/Auth/register
POST /api/Auth/login
Categories
GET /api/events/{eventId}/categories
POST /api/events/{eventId}/categories
PUT /api/categories/{id}
DELETE /api/categories/{id}
Enrolments
POST /api/categories/{categoryId}/enrolments
GET /api/enrolments/me

GitHUB


ST10493031 / RaceDay.API
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
RaceDay.API
Public
Pin
Watch 0
Fork 0
Star 0
master
1 Branch
0 Tags
Go to file
Add file
Code
ST10493031 Revise README with project updates and details
36ce80a · 5 hours ago
7 Commits
RaceDay.API.Tests Add project files. 5 hours ago
RaceDay.API Add project files. 5 hours ago
.gitattributes Add .gitattributes, .gitignore, and README.md. 5 hours ago
.gitignore Add .gitattributes, .gitignore, and README.md. 5 hours ago
PROGRAMMING 2B PROG6212 Seaney Maseko... Add files via upload 5 hours ago
RBRaceDaySQL.sql SQL 5 hours ago
README.md Revise README with project updates and details 5 hours ago
RaceDay.API.sln Add project files. 5 hours ago
workflows.txt Workflow 5 hours ago
README
RaceDay - Event Management System
Author: Seaney Maseko (ST10493031) Module: Programming 2B (PROG6212) Parts Completed: 1 (System Planning & Database) + 2 (RESTful API) Date: October 2026
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
ST10493031
Languages
C# 85.6% TSQL 8.6% HTML 4.2%
CSS 1.3% JavaScript 0.3%
Suggested workflows

Read ME

README

RaceDay - Event Management System

Author: Seaney Maseko (ST10493031) **Module:** Programming 2B (PROG6212) **Parts Completed:** 1 (System Planning & Database) + 2 (RESTful API) **Date:** October 2026

📖 Project Overview

South Africa has a rich road events culture — from the iconic Comrades Marathon to the Cape Town Cycle Tour and hundreds of community park runs. However, many of these events are still managed through paper-based registration, disconnected spreadsheets, and manual processes that overwhelm organisers and frustrate participants.

RaceDay is a full-stack web-based event management system designed specifically for the South African road running, walking, and cycling community. The platform allows **Organisers** to create and manage events, categories, and participant results, while **Participants** can browse upcoming events, enter events, and track their personal performance history.

This repository currently contains:

- **Part 1** — System planning: ERD, API endpoint plan, and SQL database schema.
- **Part 2** — A fully functional RESTful API built in C# using ASP.NET Core, with JWT authentication, role-based authorization, Entity Framework Core, Swagger documentation, and unit tests.

👥 User Roles

The system supports **two distinct user roles**, enforced at the API level through JWT claims and `[Authorize(Roles = "...")]` attributes.

🏁 Organiser

- Register and log in with an Organiser account.
- Create, edit, and delete events.
- Define age and distance categories for each event.
- View all participant enrolments for their own events.